


ESPE

UNIVERSIDAD DE LAS FUERZAS ARMADAS

INNOVACIÓN PARA LA EXCELENCIA




**DEPARTAMENTO DE TECNOLOGÍAS DE LA INFORMACIÓN**

**TEMAS**

Persistencia Poliglota - Operación Rescate

**GRUPO 4**

**INTEGRANTES:**

KEVIN LLUMIQUINGA

MATEO FLORES

JHONNY REMACHE

**NRC:** 36588

**ASIGNATURA:** MODELADO AVANZADO DE BASE DE DATOS

**FECHA:** 27/10/2026

# TALLER PERSISTENCIA POLÍGLOTA
# OPERACIÓN RESCATE

### De Monolito Colapsado a Arquitectura Políglota y Distribuida

Asignatura: Sistemas Avanzados de Base de Datos - BD Distribuidas NoSQL y Réplica
Docente: Paul Diaz Modalidad: Grupos de 3-4 estudiantes

## 1. LA HISTORIA - Caso Real

**Empresa:** "Distribuidora Valle Express" - Sangolquí. Vende electrodomésticos y abarrotes en 3 locales físicos + 1 tienda online en WordPress/WooCommerce.

**Arquitectura Actual (Colapsada):** Una única instancia MySQL 5.7 en un servidor local en la matriz. Una sola tabla gigante llamada ventas_todo con 47 columnas, donde la columna detalle_productos es un JSON gigante.

**El Viernes Negro 2025:**

- 10:00 AM: 800 usuarios simultáneos en la web. La consulta SELECT * FROM ventas_todo WHERE detalle_productos LIKE '%TV%' bloqueó la tabla por 45 segundos.
- 10:05 AM: Las cajas de los 3 locales no podían facturar porque la tabla estaba bloqueada (MySQL MyISAM). Fila de 30 clientes.
- 11:30 AM: Se cayó el servidor local por falta de espacio. El disco se llenó con logs binarios. No había réplica.
- 12:00 PM: Pérdida estimada: $18,000 en ventas + 120 clientes molestos + 3 horas de datos de inventario no registrados.
- Pregunta del Gerente al equipo de TI: '¿Por qué si tenemos una base de datos potente, todo se cae cuando más vendemos?'

## 2. OBJETIVO DEL TALLER

No van a arreglar MySQL. Van a diseñar una nueva arquitectura distribuida, políglota y con réplica que garantice que esto NUNCA vuelva a pasar. Al final del taller deben ser capaces de responder con teoría CAP, ACID/BASE y metodologías de réplica.

## 3. CONFORMACIÓN DE EQUIPOS Y ROLES

- Arquitecto/a CAP (Líder): Decide qué nivel de consistencia necesita cada dato. Justifica con

PACELC.

- DBA SQL: Responsable de diseñar la parte transaccional ACID. Define qué va en PostgreSQL y su estrategia de réplica.
- DBA NoSQL: Responsable de diseñar la parte polimórfica y de alta velocidad. Define qué va en MongoDB y Redis y su estrategia de réplica.
- Ingeniero/a de Caos (QA): Su trabajo es romper el diseño de sus compañeros. Debe proponer 3 escenarios de fallo.

## 4. FASES DEL TALLER

**FASE 1 - Autopsia**

Cada grupo debe responder en 1 página:

1. ¿Qué teorema violó la empresa? ¿Por qué una sola BD no puede ser Consistente, Disponible y Tolerante a Particiones a la vez cuando hay mucho tráfico?

   La empresa violó el teorema CAP. La empresa operaba como un sistema “CA” de único: una sola instancia MySQL, sin réplica y sin tolerancia a fallos. Eso representa sólo una ilusión, porque en el entorno práctico, las fallas de red, de disco o de saturación ocurren siempre. Al momento de llegar la carga del Viernes Negro, el sistema perdio disponibilidad:

   - 10:05 AM: el LIKE ‘%TV’ sobre la tabla “MyISAM” tomó un table lock de 45 segundos. Para proteger la consistencia, bloqueo incluso las facturas de las cajas.
   - 11:30 AM: al llenarse el disco con logs binarios, el servidor cayó y no había réplica que asumiera. Disponibilidad cero y datos de inventario perdidos.

   Bajo carga alta, los timeouts, los bloques y las caídas se comportan como una partición: los nodos o los clientes dejan de poder comunicarse con el dato. En ese momento el sistema debe elegir:

   - CP: rechaza o bloquea peticiones para no mostrar datos inconsistentes (se pierde A).
   - AP: sigue respondiendo con datos posiblemente desactualizados (se pierde C).

   No existe una tercera opción que ofrezca ambas a la vez. Además, con un solo nodo no hay réplicas entre las que elegir, así que no hay tolerancia a particiones.

   Lectura PACELC. Incluso sin falla, existe un compromiso entre latencia y consistencia. MySQL con table locks eligió consistencia a costa de latencia (comportamiento EC), y esa latencia de 45 segundos fue la que paralizó el negocio.

Violacion adicional de ACID. MyISAM no es transaccional y bloquea a nivel de tabla. Mezclar lecturas analíticas pesadas con escrituras de facturación en la misma tabla rompe el aislamiento y la concurrencia que necesita una caja registradora.

2. Identifiquen 3 tipos de datos que estaban mezclados en ventas_todo y que tienen necesidades distintas (ej: factura vs. carrito vs. búsqueda).

| Tipo de dato | Ejemplo en el caso | Necesidad principal | Modelo adecuado |
|---|---|---|---|
| Transaccional crítico | facturas, pagos, stock contable | Atomicidad e integridad, cero perdida (ACID, consistencia fuerte) | Relacional |
| Semiestructurado / polimórfico | detalle_productos (JSON grande), catálogo, clientes, logs de pedidos | Esquema flexible: un TV tiene “pulgadas” y un abarrote tiene “fecha de caducidad” | Documentos |
| Efímero y de alta velocidad | carrito, sesiones de usuario, caché de stock | Latencia de milisegundos, expiración automática (TTL), tolerancia a perder datos | Clave - valor en memoria |

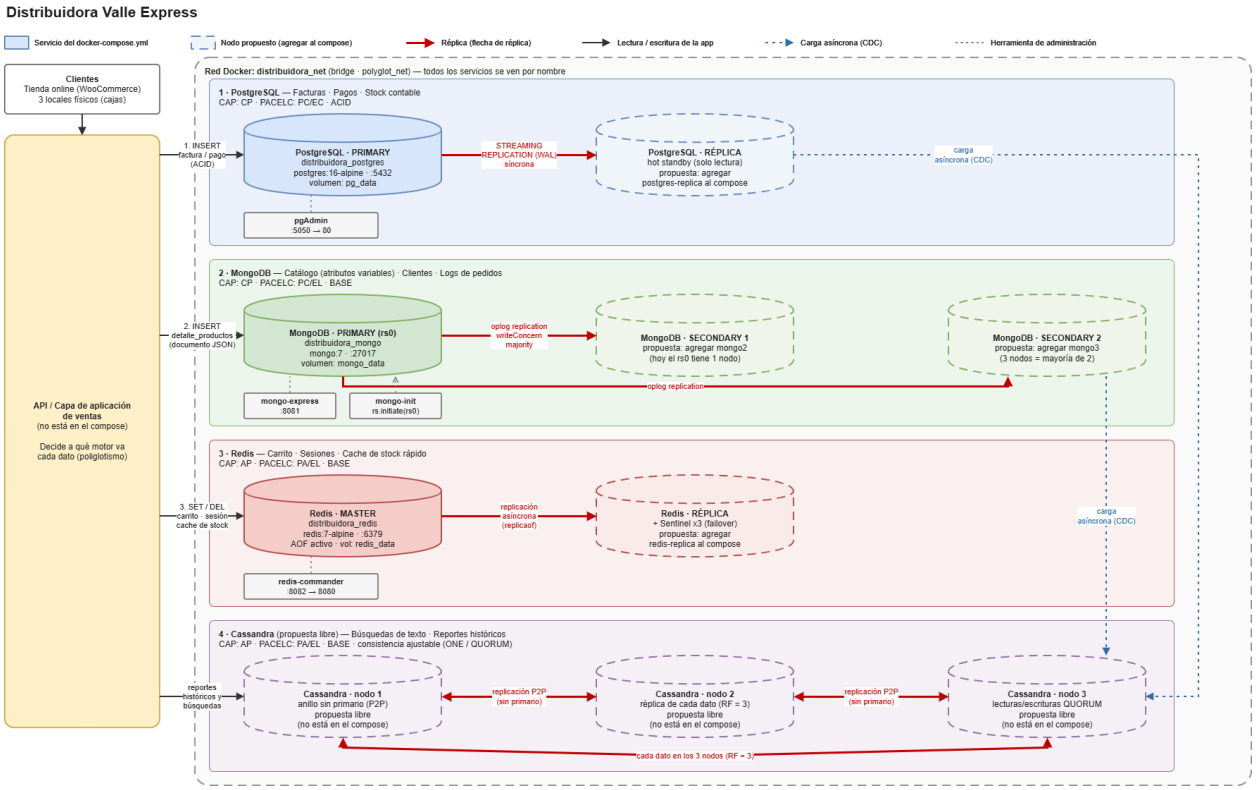
Cada tipo de dato exige un equilibrio distinto entre consistencia y disponibilidad. Una factura debe ser ACID. Un carrito puede ser BASE (Basically Available, Soft state, Eventually consistent). Mezclarlos en una tabla obliga a pagar el costo de ACID en todo.

3. ¿Qué diferencia hay entre persistencia políglota y polimórfica aplicada a este caso?

| Persistencia políglota | Persistencia polimórfica |
|---|---|
| Usa varios motores de bases de datos, cada uno para el tipo de datos que mejor maneja | Maneja datos de estructura variable dentro de un mismo motor, sin esquema fijo |
| A nivel de arquitectura (la decisión de “qué motor para qué” | A nivel de modelo de datos (cómo se guarda cada registro) |
| En este caso, las facturas en PostresSQL, catálogo en MongoDB y carrito en Redis | Para este caso, en una sola colección de MongoDB conviven un TV {marca, pulgadas, resolución} y un arroz {peso_kg, caducidad} sin ALTER TABLE |

**FASE 2 - Diseño Políglota (trabajo autónomo)**

Dibujen la nueva arquitectura. Obligatorio usar el docker-compose.yml entregado.




| Dato | Motor Propuesto | Justificación Teórica (ACID/BASE + CAP) | Metodología de Réplica |
|---|---|---|---|
| **Facturas, Pagos, Stock Contable** | **PostgreSQL** | ACID: no se puede perder ni duplicar un pago.<br><br>CAP: CP<br><br>PACELC: PC/EC: con partición prefiere rechazar antes que aceptar algo inconsistente, y sin partición acepta más latencia a cambio de consistencia. | Streaming replication a 1 hot standby, síncrona.<br><br>La réplica atiende reportes y se promueve si cae el primario. |

| | | | |
|---|---|---|---|
| **Catálogo productos (atributos variables), Clientes, Logs pedidos** | **MongoDB** | BASE: esquemas flexibles (el JSON grande de detalle_productos).<br><br>CAP: CP, porque sin mayoría no hay primario.<br><br>PACELC: PC/EL: las lecturas de catálogo van a secundarios, con menor latencia y consistencia eventual. | Replica Set de 3 nodos con “writeConcern: majority” |
| **Carrito sesión, Sesiones usuario, Cache stock rápido** | **Redis** | BASE: datos efímeros y de alta velocidad.<br><br>CAP: AP<br><br>PACELC: PA/EL: prioriza responder rápido. El stock real manda en PostreSQL. | Master + 1 réplica asíncrona + Sentinel x 3.<br><br>El compose ya activa AOF. |
| **Búsquedas texto, Reportes históricos** | **Cassandra** | BASE: modelo por consulta (tablas desnormalizadas por categoría, marca, cliente o fecha), pensado para escritura masiva histórica y lecturas rápidas por clave.<br><br>CAP: AP<br><br>PACELC: PA/EL: responde aun con partición y prioriza latencia, con consistencia ajustable (ONE o QUORUM). | Anillo de 3 nodos sin primario, replication factor 3, lecturas y escrituras QUORUM. |

Para cada fila deben llenar: ¿Por qué ese motor? (ej: ACID porque no puedo perder un pago) y ¿Qué réplica usarán? (ej: PostgreSQL Streaming Asíncrona + 1 réplica, MongoDB Replica Set 3 nodos con WriteConcern majority)

**FASE 3 - Prueba de Supervivencia (Laboratorio)**

1. Levantar el compose: docker compose up -d y verificar rs.status() en Mongo.

docker compose up -d

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poliglotismo> docker compose up -d
time="2026-10-25T22:47:25-05:00" level=warning msg="C:\\Users\\teo20\\Desktop\\U\\5_TICs\\SEMESTRE 6\\MODELADO AVANZADO DE DATOS - 36588\\PARCIAL 1\\Taller-Operacion-Rescate
-Poliglotismo\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 59/69
 - Image rediscommander/redis-commander:latest [⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀] 28.33MB / 28.33MB Pulling                                                    26.0s
 ✓ Image redis:7-alpine                                                                Pulled                                                     19.8s
 - Image mongo-express:latest [⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀] 58.74MB / 59MB                Pulling                                                    26.0s
 - Image cassandra:4.1 [⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀] 151.1MB / 152.3MB                  Pulling                                                    26.0s
 - Image dpage/pgadmin4:latest [⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀]   148MB / 185MB      Pulling                                                    26.0s
 - Image postgres:16-alpine [⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀⠀] 61.33MB / 116.7MB           Pulling                                                    26.0s
```

docker ps

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poliglotismo> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED         STATUS                            PORTS
          NAMES
da0bcff6abd8   dpage/pgadmin4:latest                "/entrypoint.sh"         19 minutes ago   Up 21 seconds                     0.0.0.0:5
          distribuidora_pgadmin
3090ca891f6f   mongo-express:latest                 "/sbin/tini -- /dock…"   19 minutes ago   Up 19 seconds                     0.0.0.0:8
cp        distribuidora_mongo_express
93426929d6b3   rediscommander/redis-commander:latest   "/usr/bin/dumb-init …"   19 minutes ago   Up 21 seconds (health: starting)   0.0.0.0:8
cp        distribuidora_redis_commander
83bcf7830f7c   postgres:16-alpine                   "docker-entrypoint.s…"   19 minutes ago   Up 32 seconds (healthy)           0.0.0.0:5
cp        distribuidora_postgres
1818c53a4b74   redis:7-alpine                       "docker-entrypoint.s…"   19 minutes ago   Up 32 seconds (healthy)           0.0.0.0:6
cp        distribuidora_redis
f3b1c4e5bb4a   cassandra:4.1                        "docker-entrypoint.s…"   19 minutes ago   Up 32 seconds (healthy)           0.0.0.0:9
cp        distribuidora_cassandra
a6f4b452a0a3   mongo:7                              "docker-entrypoint.s…"   19 minutes ago   Up 32 seconds (healthy)           0.0.0.0:2
17/tcp    distribuidora_mongo
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poliglotismo> s
```

docker exec -it distribuidora_mongo mongosh --eval "rs.status()"

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Oper
 --eval "rs.status()"
    {
      _id: 0,
      name: 'distribuidora_mongo:27017',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 18,
      optime: { ts: Timestamp({ t: 1793022494, i: 1 }), t: Long('3') },
      optimeDate: ISODate('2026-10-26T13:48:14.000Z'),
      lastAppliedWallTime: ISODate('2026-10-26T13:48:14.140Z'),
      lastDurableWallTime: ISODate('2026-10-26T13:48:14.140Z'),
      syncSourceHost: '',
```

**Nota:** Salida de rs.status() mostrando el nodo como PRIMARY

2. Insertar un caso de uso real distribuido: Una venta. Hacer: INSERT en Postgres (factura), INSERT en Mongo (detalle productos), SET en Redis (vaciar carrito).

   a) PostgreSQL – Factura (ACID, CP)

   docker exec -it distribuidora_postgres psql -U admin -d distribuidora_acid

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARC
  -U admin -d distribuidora_acid
psql (16.15)
Type "help" for help.

distribuidora_acid=# CREATE TABLE IF NOT EXISTS facturas (
distribuidora_acid(#     id SERIAL PRIMARY KEY,
distribuidora_acid(#     cliente_id INT,
distribuidora_acid(#     total NUMERIC(10,2),
distribuidora_acid(#     fecha TIMESTAMP DEFAULT NOW()
distribuidora_acid(# );
CREATE TABLE
distribuidora_acid=# INSERT INTO facturas (cliente_id, total) VALUES (101, 250.75);
INSERT 0 1
distribuidora_acid=# SELECT * FROM facturas;
 id | cliente_id | total  |           fecha
----+------------+--------+----------------------------
  1 |        101 | 250.75 | 2026-10-26 13:52:47.260856
(1 row)
```

```
distribuidora_acid=# SELECT * FROM facturas;
 id | cliente_id | total  |           fecha
----+------------+--------+----------------------------
  1 |        101 | 250.75 | 2026-10-26 13:52:47.260856
(1 row)

distribuidora_acid=#
```

**Nota:** SELECT * FROM facturas; con la fila insertada.

b) MongoDB – Detalle de productos (polimórfico, CP con majority)

docker exec -it distribuidora_mongo mongosh

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poliglotismo> docker exec -it distribuid
ora_mongo mongosh
Current Mongosh Log ID: 6adf5bd01122b3807f26b81f
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.12.0
Using MongoDB:          7.0.43
Using Mongosh:          2.12.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, pseudo-anonymous usage data is collected from interactive and agentic sessions only, and sent to MongoDB periodically (https:/
/www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-10-26T13:48:03.239+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/pr
odnotes-filesystem
```

```
rs0 [direct: primary] distribuidora_catalogo;> use distribuidora_catalogo;
switched to db distribuidora_catalogo
rs0 [direct: primary] distribuidora_catalogo> db.detalle_productos.insertOne({
|   factura_id: 1,
|   productos: [
|     { nombre: "TV Samsung 55\"", atributos: { pulgadas: 55, resolucion: "4K" } },
|     { nombre: "Arroz 5kg", atributos: { peso_kg: 5, caducidad: "2026-12-31" } }
|   ]
| });
{
  acknowledged: true,
  insertedId: ObjectId('6adf5c5d1122b3807f26b820')
}
```

```
rs0 [direct: primary] distribuidora_catalogo> db.detalle_productos.find();
|
[
  {
    _id: ObjectId('6adf5c5d1122b3807f26b820'),
    factura_id: 1,
    productos: [
      {
        nombre: 'TV Samsung 55"',
        atributos: { pulgadas: 55, resolucion: '4K' }
      },
      {
        nombre: 'Arroz 5kg',
        atributos: { peso_kg: 5, caducidad: '2026-12-31' }
      }
    ]
  }
]
rs0 [direct: primary] distribuidora_catalogo>
```

c) Redis – Carrito (BASE, AP)

docker exec -it distribuidora_redis redis-cli -a Admin123

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Ope
ora_redis redis-cli -a Admin123
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
127.0.0.1:6379>
```

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODEL
ora_redis redis-cli -a Admin123
Warning: Using a password with '-a' or '-u' option
127.0.0.1:6379> SET carrito:usuario:101 "TV,Arroz"
OK
127.0.0.1:6379> GET carrito:usuario:101
"TV,Arroz"
127.0.0.1:6379> DEL carrito:usuario:101
(integer) 1
127.0.0.1:6379> GET carrito:usuario:101
(nil)
127.0.0.1:6379>
```

**Nota:** GET carrito:usuario:101 antes y después del DEL.

d) Cassandra – Registro de búsqueda/reporte histórico (BASE, AP)

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL
ora_cassandra cqlsh

ATTENTION: All commands will be saved to history file: /root/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use --disable-history or set 'disabled = true' in the [history] secti
See https://cassandra.apache.org/doc/latest/tools/cqlsh.html for more information.

Connected to DistribuidoraCluster at 127.0.0.1:9042
[cqlsh 6.1.0 | Cassandra 4.1.12 | CQL spec 3.4.6 | Native protocol v5]
Use HELP for help.
cqlsh> CREATE KEYSPACE IF NOT EXISTS distribuidora_reportes
   ... WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
cqlsh> USE distribuidora_reportes;
cqlsh:distribuidora_reportes> CREATE TABLE IF NOT EXISTS busquedas (
                          ...     id UUID PRIMARY KEY,
                          ...     termino TEXT,
                          ...     fecha TIMESTAMP,
                          ...     cliente_id INT
                          ... );
cqlsh:distribuidora_reportes> INSERT INTO busquedas (id, termino, fecha, cliente_id)
                          ... VALUES (uuid(), 'TV Samsung', toTimestamp(now()), 101);
```

```
cqlsh:distribuidora_reportes> SELECT * FROM busquedas;

 id                                   | cliente_id | fecha                           | termino
--------------------------------------+------------+---------------------------------+------------
 708d3749-f3aa-4779-bb34-7f7bd993c936 |        101 | 2026-10-26 14:02:43.215000+0000 | TV Samsung

(1 rows)
cqlsh:distribuidora_reportes>
```

**Nota:** SELECT * FROM busquedas; mostrando el registro.

3. Ejecutar el Plan de Caos del Ingeniero de Caos: a) docker stop distribuidora_postgres -> ¿Sigue el catálogo en Mongo? b) docker stop distribuidora_mongo -> ¿Siguen facturando? c) Simular lag: ¿Qué pasa si Redis se cae?

**PASO 1 – Insertar datos de prueba en los 4 motores**

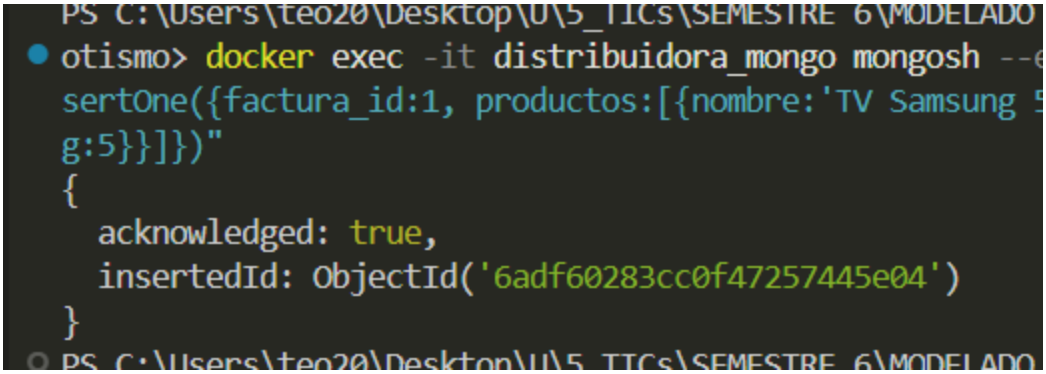
**1.1 PostgreSQL – factura**

```
docker exec -it distribuidora_postgres psql -U admin -d distribuidora_acid -c
"CREATE TABLE IF NOT EXISTS facturas (id SERIAL PRIMARY KEY, cliente_id INT,
total NUMERIC(10,2), fecha TIMESTAMP DEFAULT NOW()); INSERT INTO facturas
(cliente_id, total) VALUES (101, 250.75);"
```

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO
otismo> docker exec -it distribuidora_postgres psql -U
 (id SERIAL PRIMARY KEY, cliente_id INT, total NUMERIC
e_id, total) VALUES (101, 250.75);"
NOTICE:  relation "facturas" already exists, skipping
CREATE TABLE
INSERT 0 1
```

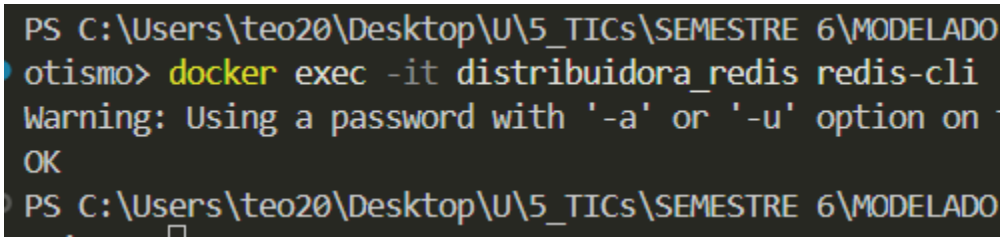
**1.2 MongoDB – detalle de productos**

```
docker exec -it distribuidora_mongo mongosh --eval
"db.getSiblingDB('distribuidora_catalogo').detalle_productos.insertOne({factur
a_id:1, productos:[{nombre:'TV Samsung 55',
atributos:{pulgadas:55}},{nombre:'Arroz 5kg', atributos:{peso_kg:5}}]})"
```



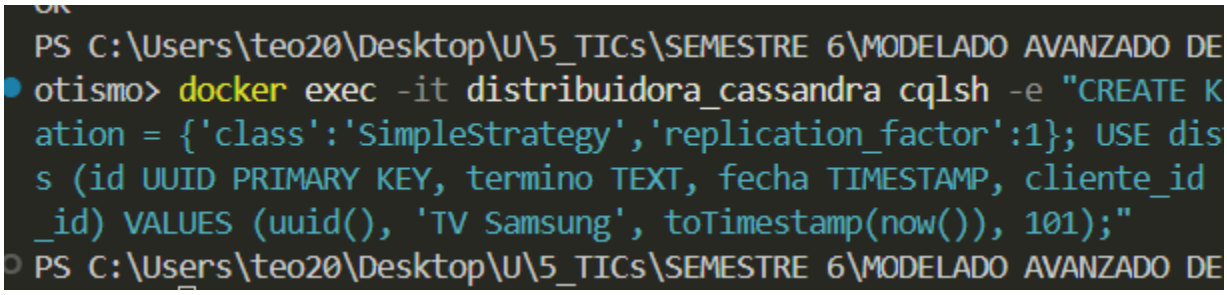

**1.3 Redis – carrito**

```
docker exec -it distribuidora_redis redis-cli -a Admin123 SET
carrito:usuario:101 "TV,Arroz"
```




**1.4 Cassandra – búsqueda/reporte histórico**

```
docker exec -it distribuidora_cassandra cqlsh -e "CREATE KEYSPACE IF NOT
EXISTS distribuidora_reportes WITH replication =
{'class':'SimpleStrategy','replication_factor':1}; USE distribuidora_reportes;
CREATE TABLE IF NOT EXISTS busquedas (id UUID PRIMARY KEY, termino TEXT, fecha
TIMESTAMP, cliente_id INT); INSERT INTO busquedas (id, termino, fecha,
cliente_id) VALUES (uuid(), 'TV Samsung', toTimestamp(now()), 101);"
```




**1.5 Verificación inicial: los 4 motores responden**

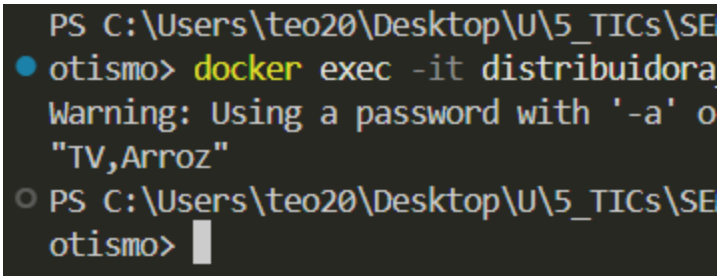
```
docker exec -it distribuidora_postgres psql -U admin -d distribuidora_acid -c
"SELECT * FROM facturas;"
```

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO A
otismo> docker exec -it distribuidora_postgres psql -U a
 id | cliente_id | total  |           fecha
----+------------+--------+----------------------------
  1 |        101 | 250.75 | 2026-10-26 13:52:47.260856
  2 |        101 | 250.75 | 2026-10-26 14:11:29.034017
(2 rows)
```

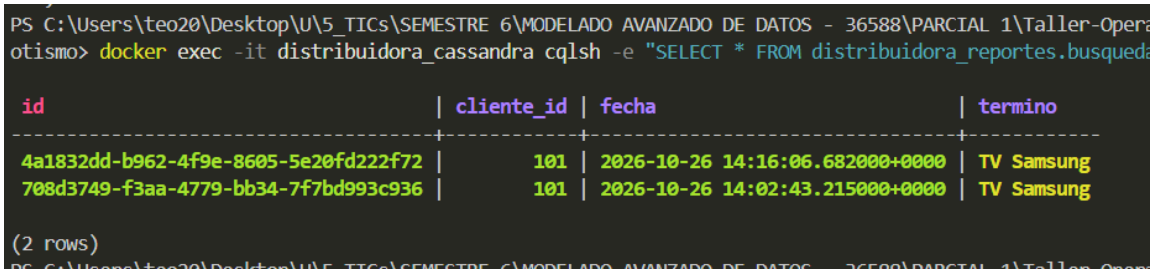
```
docker exec -it distribuidora_mongo mongosh --eval
"db.getSiblingDB('distribuidora_catalogo').detalle_productos.find()"
```

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO
otismo> docker exec -it distribuidora_mongo mongosh --eval "db.ge
nd()"
[
  {
    _id: ObjectId('6adf5c5d1122b3807f26b820'),
    factura_id: 1,
    productos: [
      {
        nombre: 'TV Samsung 55"',
        atributos: { pulgadas: 55, resolucion: '4K' }
      },
      {
        nombre: 'Arroz 5kg',
        atributos: { peso_kg: 5, caducidad: '2026-12-31' }
      }
    ]
  },
  {
    _id: ObjectId('6adf60283cc0f47257445e04'),
    factura_id: 1,
    productos: [
      { nombre: 'TV Samsung 55', atributos: { pulgadas: 55 } },
      { nombre: 'Arroz 5kg', atributos: { peso_kg: 5 } }
    ]
  }
]
```

```
docker exec -it distribuidora_redis redis-cli -a Admin123 GET
carrito:usuario:101
```




```
docker exec -it distribuidora_cassandra cqlsh -e "SELECT * FROM
distribuidora_reportes.busquedas;"
```

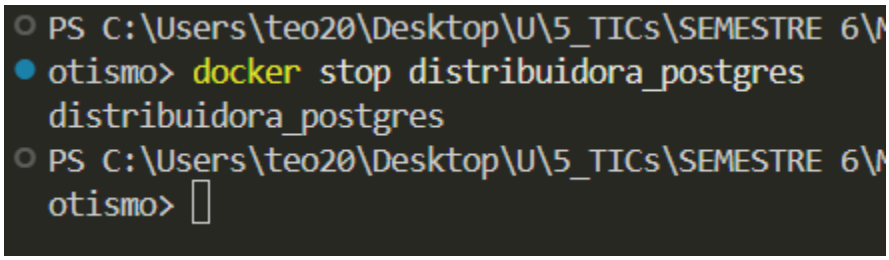



**ESCENARIO a) ¿Qué pasa si se cae PostgreSQL? → ¿Sigue el catálogo en Mongo?**

Objetivo: demostrar que si el motor transaccional (Postgres) cae, el catálogo (Mongo), el carrito (Redis) y los reportes (Cassandra) siguen operando. El negocio pierde la facturación, pero la tienda puede seguir mostrando productos.

Paso a.1 — Detener Postgres

```
docker stop distribuidora_postgres
```




Paso a.2 — Verificar que el catálogo en Mongo sigue vivo

```
docker exec -it distribuidora_mongo mongosh --eval
"db.getSiblingDB('distribuidora_catalogo').detalle_productos.find()"
```

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO
otismo> docker exec -it distribuidora_mongo mongosh --eval "db.g
nd()"
[
  {
    _id: ObjectId('6adf5c5d1122b3807f26b820'),
    factura_id: 1,
    productos: [
      {
        nombre: 'TV Samsung 55"',
        atributos: { pulgadas: 55, resolucion: '4K' }
      },
      {
        nombre: 'Arroz 5kg',
        atributos: { peso_kg: 5, caducidad: '2026-12-31' }
      }
    ]
  },
  {
    _id: ObjectId('6adf60283cc0f47257445e04'),
    factura_id: 1,
    productos: [
      { nombre: 'TV Samsung 55', atributos: { pulgadas: 55 } },
      { nombre: 'Arroz 5kg', atributos: { peso_kg: 5 } }
    ]
  }
]
```

Paso a.3 — Verificar que Redis y Cassandra también siguen vivos

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poli
otismo> docker exec -it distribuidora_redis redis-cli -a Admin123 GET carrito:usuario:101
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
"TV,Arroz"
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poli
otismo> docker exec -it distribuidora_cassandra cqlsh -e "SELECT * FROM distribuidora_reportes.busquedas;"

 id                                   | cliente_id | fecha                           | termino
--------------------------------------+------------+---------------------------------+------------
 4a1832dd-b962-4f9e-8605-5e20fd222f72 |        101 | 2026-10-26 14:16:06.682000+0000 | TV Samsung
 708d3749-f3aa-4779-bb34-7f7bd993c936 |        101 | 2026-10-26 14:02:43.215000+0000 | TV Samsung

(2 rows)
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poli
otismo> docker ps
CONTAINER ID   IMAGE                                    COMMAND                  CREATED        STATUS
ORTS                                       NAMES
da0bcff6abd8   dpage/pgadmin4:latest                    "/entrypoint.sh"         11 hours ago   Up 51 minutes
.0.0.0:5050->80/tcp, [::]:5050->80/tcp     distribuidora_pgadmin
3090ca891f6f   mongo-express:latest                     "/sbin/tini -- /dock…"   11 hours ago   Up 51 minutes
.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp distribuidora_mongo_express
93426929d6b3   rediscommander/redis-commander:latest    "/usr/bin/dumb-init …"   11 hours ago   Up 51 minutes (healthy)
.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp distribuidora_redis_commander
1818c53a4b74   redis:7-alpine                           "docker-entrypoint.s…"   11 hours ago   Up 51 minutes (healthy)
.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp distribuidora_redis
f3b1c4e5bb4a   cassandra:4.1                            "docker-entrypoint.s…"   11 hours ago   Up 51 minutes (healthy)
.0.0.0:9042->9042/tcp, [::]:9042->9042/tcp distribuidora_cassandra
a6f4b452a0a3   mongo:7                                  "docker-entrypoint.s…"   11 hours ago   Up 51 minutes (healthy)
.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp   distribuidora_mongo
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poli
otismo>
```

Paso a.4 — Confirmar que la facturación sí está caída

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Pol
otismo> docker exec -it distribuidora_postgres psql -U admin -d distribuidora_acid -c "SELECT * FROM facturas;"
Error response from daemon: container 83bcf7830f7c118aa6c84ccb5a35ac567839d7ecd8882feb99f58f6e25b87c5b is not running
```

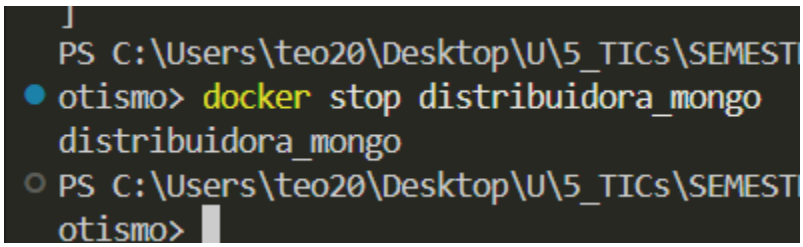
Paso a.5 — Restaurar Postgres

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Pol
otismo> docker start distribuidora_postgres
distribuidora_postgres
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Pol
otismo> Start-Sleep -Seconds 12
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Pol
otismo> docker ps
CONTAINER ID   IMAGE                                    COMMAND                  CREATED        STATUS
ORTS                                        NAMES
da0bcff6abd8   dpage/pgadmin4:latest                    "/entrypoint.sh"         11 hours ago   Up 55 minutes
.0.0.0:5050->80/tcp, [::]:5050->80/tcp      distribuidora_pgadmin
3090ca891f6f   mongo-express:latest                     "/sbin/tini -- /dock…"   11 hours ago   Up 55 minutes
.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp  distribuidora_mongo_express
93426929d6b3   rediscommander/redis-commander:latest     "/usr/bin/dumb-init …"   11 hours ago   Up 55 minutes (healthy)
.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp  distribuidora_redis_commander
83bcf7830f7c   postgres:16-alpine                       "docker-entrypoint.s…"   11 hours ago   Up 28 seconds (healthy)
.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp  distribuidora_postgres
1818c53a4b74   redis:7-alpine                           "docker-entrypoint.s…"   11 hours ago   Up 55 minutes (healthy)
.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp  distribuidora_redis
f3b1c4e5bb4a   cassandra:4.1                            "docker-entrypoint.s…"   11 hours ago   Up 55 minutes (healthy)
.0.0.0:9042->9042/tcp, [::]:9042->9042/tcp  distribuidora_cassandra
a6f4b452a0a3   mongo:7                                  "docker-entrypoint.s…"   11 hours ago   Up 55 minutes (healthy)
.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp   distribuidora_mongo
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Pol
otismo> docker exec -it distribuidora_postgres psql -U admin -d distribuidora_acid -c "SELECT * FROM facturas;"
 id | cliente_id | total  |           fecha
----+------------+--------+----------------------------
  1 |        101 | 250.75 | 2026-10-26 13:52:47.260856
  2 |        101 | 250.75 | 2026-10-26 14:11:29.034017
(2 rows)
```

ESCENARIO b) ¿Qué pasa si se cae MongoDB? → ¿Siguen facturando?

Objetivo: demostrar que si el catálogo (Mongo) cae, la facturación (Postgres), el carrito (Redis) y los reportes (Cassandra) siguen operando. El negocio puede facturar, pero no puede mostrar ni buscar productos nuevos.

Paso b.1 — Detener Mongo




Paso b.2 — Verificar que la facturación sigue viva

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poligl
otismo> docker exec -it distribuidora_postgres psql -U admin -d distribuidora_acid -c "SELECT * FROM facturas;"
 id | cliente_id | total  |          fecha
----+------------+--------+----------------------------
  1 |        101 | 250.75 | 2026-10-26 13:52:47.260856
  2 |        101 | 250.75 | 2026-10-26 14:11:29.034017
(2 rows)

PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poligl
otismo> docker exec -it distribuidora_postgres psql -U admin -d distribuidora_acid -c "INSERT INTO facturas (cliente_id, t
otal) VALUES (202, 99.90); SELECT * FROM facturas;"
INSERT 0 1
 id | cliente_id | total  |          fecha
----+------------+--------+----------------------------
  1 |        101 | 250.75 | 2026-10-26 13:52:47.260856
  2 |        101 | 250.75 | 2026-10-26 14:11:29.034017
  3 |        202 |  99.90 | 2026-10-26 14:46:54.265155
(3 rows)

PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poligl
otismo>
```

Paso b.3 — Verificar que Redis y Cassandra siguen vivos

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacior
otismo> docker exec -it distribuidora_redis redis-cli -a Admin123 GET carrito:usuario:101
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
"TV,Arroz"
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacior
otismo> docker exec -it distribuidora_cassandra cqlsh -e "SELECT * FROM distribuidora_reportes.busquedas;"

 id                                   | cliente_id | fecha                           | termino
--------------------------------------+------------+---------------------------------+------------
 4a1832dd-b962-4f9e-8605-5e20fd222f72 |        101 | 2026-10-26 14:16:06.682000+0000 | TV Samsung
 708d3749-f3aa-4779-bb34-7f7bd993c936 |        101 | 2026-10-26 14:02:43.215000+0000 | TV Samsung

(2 rows)
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacior
```

Paso b.4 — Confirmar que el catálogo sí está caído

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poligl
otismo> docker exec -it distribuidora_mongo mongosh --eval "db.getSiblingDB('distribuidora_catalogo').detalle_productos.fi
nd()"
Error response from daemon: container a6f4b452a0a393229d0806d8c3449a097b5fc1a869b8dd5533cb5526251bb70e is not running
```

Paso b.5 — Restaurar Mongo

```
]
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36
otismo> docker stop distribuidora_mongo
distribuidora_mongo
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36
otismo> docker start distribuidora_mongo
distribuidora_mongo
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36
otismo> Start-Sleep -Seconds 12
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36
otismo> docker exec -it distribuidora_mongo mongosh --eval "rs.status()"
{
```

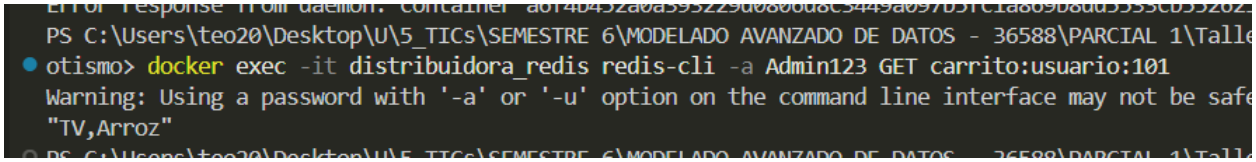
```
  },
  members: [
    {
      _id: 0,
      name: 'distribuidora_mongo:27017',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 38,
      optime: { ts: Timestamp({ t: 1793026256, i: 1 }), t: Long('4') },
      optimeDate: ISODate('2026-10-26T14:50:56.000Z'),
      lastAppliedWallTime: ISODate('2026-10-26T14:50:56.680Z'),
      lastDurableWallTime: ISODate('2026-10-26T14:50:56.680Z'),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: 'Could not find member to sync from',
      electionTime: Timestamp({ t: 1793026226, i: 1 }),
      electionDate: ISODate('2026-10-26T14:50:26.000Z'),
      configVersion: 1,
      configTerm: 4,
      self: true,
      lastHeartbeatMessage: ''
    }
  ],
  ok: 1,
```



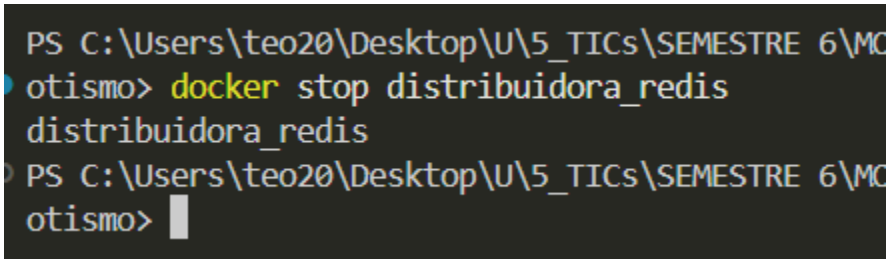

ESCENARIO c) Simular lag / caída de Redis → ¿Qué pasa?

Objetivo: demostrar que Redis es un motor efímero y prescindible. Si cae, se pierden los carritos activos y la caché, pero la facturación (Postgres) y el catálogo (Mongo) no se ven afectados. Es el caso típico de "lag" o pérdida de estado en memoria.

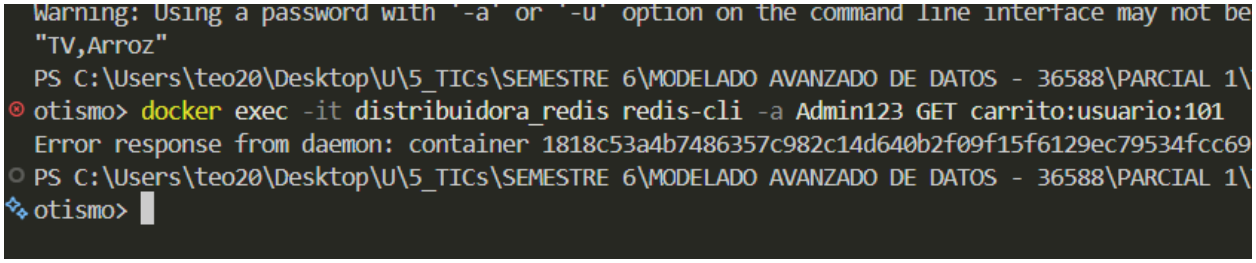
Paso c.1 — Verificar el carrito ANTES de la caída




Paso c.2 — Detener Redis




Paso c.3 — Intentar leer el carrito con Redis caído




Paso c.4 — Verificar que el resto del negocio sigue operando

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operac
smo> docker exec -it distribuidora_postgres psql -U admin -d distribuidora_acid -c "SELECT * FROM factura
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operac
smo> docker exec -it distribuidora_mongo mongosh --eval "db.getSiblingDB('distribuidora_catalogo').detal
[
  {
    _id: ObjectId('6adf5c5d1122b3807f26b820'),
    factura_id: 1,
    productos: [
      {
        nombre: 'TV Samsung 55"',
        atributos: { pulgadas: 55, resolucion: '4K' }
      },
      {
        nombre: 'Arroz 5kg',
        atributos: { peso_kg: 5, caducidad: '2026-12-31' }
      }
    ]
  },
  {
    _id: ObjectId('6adf60283cc0f47257445e04'),
    factura_id: 1,
    productos: [
      { nombre: 'TV Samsung 55', atributos: { pulgadas: 55 } },
      { nombre: 'Arroz 5kg', atributos: { peso_kg: 5 } }
    ]
  }
]
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operac
smo> docker exec -it distribuidora_cassandra cqlsh -e "SELECT * FROM distribuidora_reportes.busquedas;"

 id                                   | cliente_id | fecha                           | termino
--------------------------------------+------------+---------------------------------+------------
 4a1832dd-b962-4f9e-8605-5e20fd222f72 |        101 | 2026-10-26 14:16:06.682000+0000 | TV Samsung
 708d3749-f3aa-4779-bb34-7f7bd993c936 |        101 | 2026-10-26 14:02:43.215000+0000 | TV Samsung
```

Paso c.5 — Restaurar Redis

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poliglo
smo>    docker start distribuidora_redis
distribuidora_redis
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poliglo
smo> Start-Sleep -Seconds 10
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poliglo
smo> docker ps
CONTAINER ID   IMAGE                                      COMMAND                  CREATED        STATUS
ORTS                                       NAMES
da0bcff6abd8   dpage/pgadmin4:latest                      "/entrypoint.sh"         11 hours ago   Up About an hour
.0.0.0:5050->80/tcp, [::]:5050->80/tcp     distribuidora_pgadmin
3090ca891f6f   mongo-express:latest                       "/sbin/tini -- /dock…"   11 hours ago   Up About an hour
.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp distribuidora_mongo_express
93426929d6b3   rediscommander/redis-commander:latest      "/usr/bin/dumb-init …"   11 hours ago   Up About an hour (healthy)
.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp distribuidora_redis_commander
83bcf7830f7c   postgres:16-alpine                         "docker-entrypoint.s…"   11 hours ago   Up 15 minutes (healthy)
.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp distribuidora_postgres
1818c53a4b74   redis:7-alpine                             "docker-entrypoint.s…"   11 hours ago   Up 20 seconds (healthy)
.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp distribuidora_redis
f3b1c4e5bb4a   cassandra:4.1                              "docker-entrypoint.s…"   11 hours ago   Up About an hour (healthy)
.0.0.0:9042->9042/tcp, [::]:9042->9042/tcp distribuidora_cassandra
a6f4b452a0a3   mongo:7                                    "docker-entrypoint.s…"   11 hours ago   Up 8 minutes (healthy)
.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp   distribuidora_mongo
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poliglo
```

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Ope
smo> docker exec -it distribuidora_redis redis-cli -a Admin123 GET carrito:usuario:101
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
"TV,Arroz"
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Ope
```

ESCENARIO d) ¿Qué pasa si se cae Cassandra?

Objetivo: demostrar que si el motor de reportes/búsquedas históricas cae, la operación transaccional y el catálogo siguen intactos. Coherente con que Cassandra es AP/BASE y solo se usa para analítica.

Paso d.1 — Detener Cassandra

```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVAN
smo> docker stop distribuidora_cassandra
distribuidora_cassandra
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVAN
smo>
```

Paso d.2 — Verificar que facturación, catálogo y carrito siguen vivos

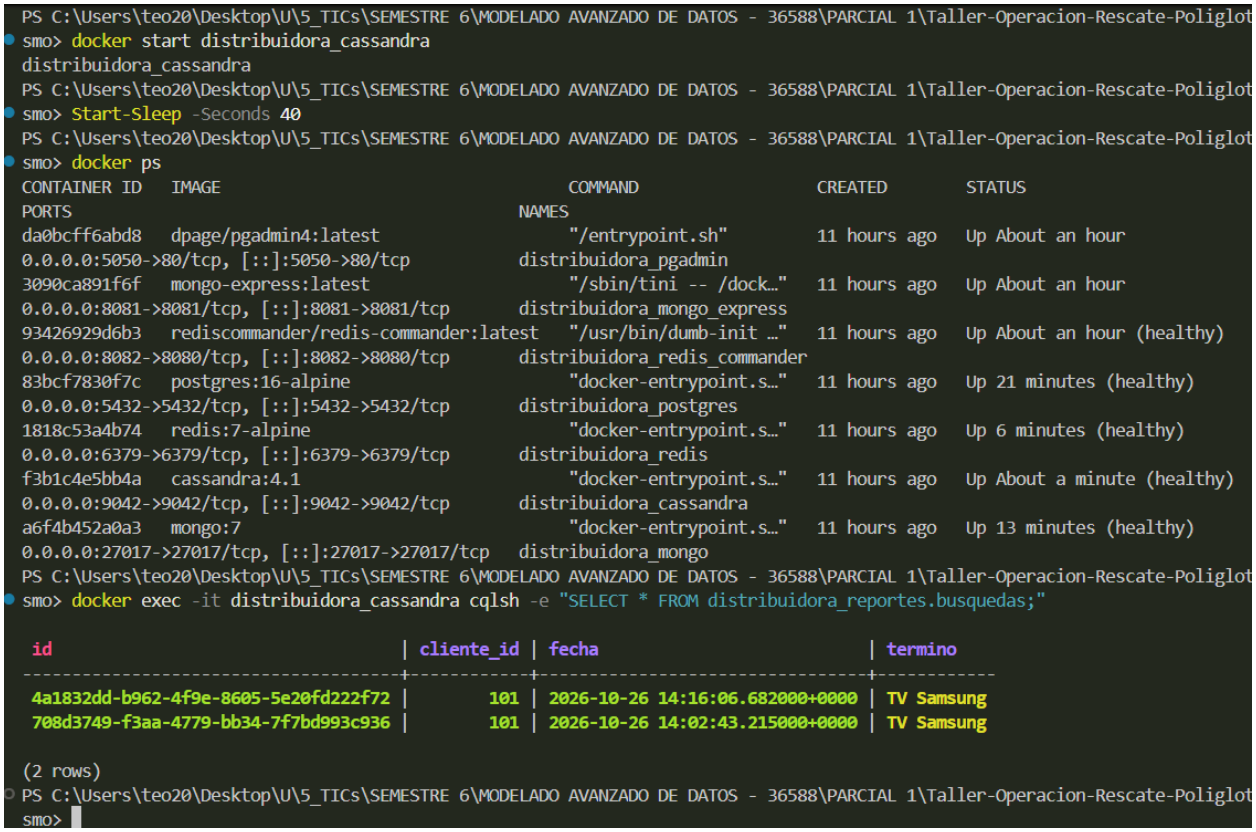
```
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Resc
smo> docker exec -it distribuidora_postgres psql -U admin -d distribuidora_acid -c "SELECT * FROM facturas;"
 id | cliente_id | total  |           fecha
----+------------+--------+----------------------------
  1 |        101 | 250.75 | 2026-10-26 13:52:47.260856
  2 |        101 | 250.75 | 2026-10-26 14:11:29.034017
  3 |        202 |  99.90 | 2026-10-26 14:46:54.265155
(3 rows)

PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Resc
smo> docker exec -it distribuidora_redis redis-cli -a Admin123 GET carrito:usuario:101
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
"TV,Arroz"
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Resc
smo> docker exec -it distribuidora_mongo mongosh --eval "db.getSiblingDB('distribuidora_catalogo').detalle_produ
[
  {
    _id: ObjectId('6adf5c5d1122b3807f26b820'),
    factura_id: 1,
    productos: [
      {
        nombre: 'TV Samsung 55"',
        atributos: { pulgadas: 55, resolucion: '4K' }
      },
      {
        nombre: 'Arroz 5kg',
        atributos: { peso_kg: 5, caducidad: '2026-12-31' }
      }
    ]
  },
  {
    _id: ObjectId('6adf60283cc0f47257445e04'),
    factura_id: 1,
```

Paso d.3 — Confirmar que los reportes están caídos

```
]
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poli
smo> docker exec -it distribuidora_cassandra cqlsh -e "SELECT * FROM distribuidora_reportes.busquedas;"
Error response from daemon: container f3b1c4e5bb4aae451feaef17b8dbdb4410f219be1020df6f930e4ea4a053f301 is not running
PS C:\Users\teo20\Desktop\U\5_TICs\SEMESTRE 6\MODELADO AVANZADO DE DATOS - 36588\PARCIAL 1\Taller-Operacion-Rescate-Poli
smo>
```

Paso d.4 — Restaurar Cassandra




| # | Escenario | Comando ejecutado | Servicio objetivo | Resultado observado | Conclusión |
|---|---|---|---|---|---|
| **0** | Verificación inicial | docker ps | Todos | 4 motores Up / healthy | Entorno listo |
| **0** | Verificación Replica Set | docker exec -it distribuidora_mongo mongosh --eval "rs.status()" | MongoDB | "ok": 1, stateStr: 'PRIMARY', set: 'rs0' | Replica Set operativo |
| **0** | Inserción venta | INSERT en Postgres + insertOne en Mongo + SET en Redis + INSERT en Cassandra | 4 motores | 2–3 filas/docs visibles en cada motor | Caso de uso distribuido insertado |

| | | | | | |
|---|---|---|---|---|---|
| **a** | Postgres caído | docker stop distribuidora_postgres | PostgreSQL | Contenedor detenido | — |
| **a.1** | Verificar vivos | redis-cli GET carrito:usuario:101 | Redis | "TV,Arroz" | Redis sigue disponible |
| **a.2** | Verificar vivos | cqlsh -e "SELECT * FROM distribuidora_reportes.busquedas;" | Cassandra | 2 filas devueltas | Cassandra sigue disponible |
| **a.3** | Verificar vivos | docker ps | Mongo / Redis / Cassandra | Los 3 healthy; Postgres ausente | Catálogo y carrito siguen operando |
| **a.4** | Verificar caído | psql -c "SELECT * FROM facturas;" | PostgreSQL | container ... is not running | **No se puede facturar** |
| **a.5** | Restaurar | docker start distribuidora_postgres + Start-Sleep -Seconds 12 | PostgreSQL | Up (healthy) | Postgres recuperado |
| **b** | Mongo caído | docker stop distribuidora_mongo | MongoDB | Contenedor detenido | — |
| **b.1** | Verificar vivos | psql -c "INSERT INTO facturas ...; SELECT * FROM facturas;" | PostgreSQL | INSERT 0 1 + 3 filas listadas | **Sí siguen facturando** |
| **b.2** | Verificar vivos | redis-cli GET carrito:usuario:101 | Redis | "TV,Arroz" | Redis sigue disponible |
| **b.3** | Verificar vivos | cqlsh -e "SELECT * FROM distribuidora_reportes.busquedas;" | Cassandra | 2 filas devueltas | Cassandra sigue |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | disponible |
| **b.4** | Verificar caído | mongosh --eval "db...detalle_productos.find()" | MongoDB | container ... is not running | **No se ve el catálogo** |
| **b.5** | Restaurar | docker start distribuidora_mongo + Start-Sleep -Seconds 12 + rs.status() | MongoDB | "ok": 1, PRIMARY (nuevo term: 4) | Mongo recuperado y reelecto |
| **c** | Redis caído | docker stop distribuidora_redis | Redis | Contenedor detenido | — |
| **c.1** | Verificar caído | redis-cli GET carrito:usuario:101 | Redis | container ... is not running | **Carrito no disponible** |
| **c.2** | Verificar vivos | psql -c "SELECT * FROM facturas;" | PostgreSQL | 3 filas listadas | Facturación intacta |
| **c.3** | Verificar vivos | mongosh --eval "db...detalle_productos.find()" | MongoDB | 2 documentos devueltos | Catálogo intacto |
| **c.4** | Verificar vivos | cqlsh -e "SELECT * FROM distribuidora_reportes.busquedas;" | Cassandra | 2 filas devueltas | Reportes intactos |
| **c.5** | Restaurar | docker start distribuidora_redis + Start-Sleep -Seconds 10 | Redis | Up (healthy) | Redis recuperado |
| **c.6** | Verificar AOF | redis-cli GET carrito:usuario:101 | Redis | "TV,Arroz" | AOF recuperó el estado persistido |

| | | | | | |
|---|---|---|---|---|---|
| **d** | Cassandra caído | docker stop distribuidora_cassandra | Cassandra | Contenedor detenido | — |
| **d.1** | Verificar caído | cqlsh -e "SELECT * FROM distribuidora_reportes.busquedas;" | Cassandra | container ... is not running | **Reportes no disponibles** |
| **d.2** | Verificar vivos | psql -c "SELECT * FROM facturas;" | PostgreSQL | 3 filas listadas | Facturación intacta |
| **d.3** | Verificar vivos | mongosh --eval "db...detalle_productos.find()" | MongoDB | 2 documentos devueltos | Catálogo intacto |
| **d.4** | Verificar vivos | redis-cli GET carrito:usuario:101 | Redis | "TV,Arroz" | Carrito intacto |
| **d.5** | Restaurar | docker start distribuidora_cassandra + Start-Sleep -Seconds 40 | Cassandra | Up (healthy) | Cassandra recuperado |
| **Final** | Confirmación | docker ps | Todos | 4 motores Up (healthy) | Sistema restaurado |

**Conclusión general de las pruebas**

- Ningún motor es un punto único de fallo total. Cada caída aislada deja operativos a los otros 3 motores.
- Postgres (ACID/CP) es el único que, al caer, bloquea la facturación. Por eso necesita réplica en producción (streaming síncrona a hot standby).
- MongoDB (CP, Replica Set) al caer bloquea el catálogo, pero la facturación continúa. En producción, con 3 nodos, un secundario asumiría como primario.
- Redis (AP/BASE) al caer solo afecta el carrito (dato efímero). El AOF del compose permite recuperar el último estado.
- Cassandra (AP/BASE) al caer solo afecta reportes/búsquedas. No impacta la operación transaccional ni el catálogo.
- La arquitectura políglota aísla los fallos y permite que el negocio siga operando parcialmente ante la caída de cualquier motor, algo imposible con el monolito MySQL original.

4. Documentar en una tabla: Qué se cayó, Qué siguió funcionando, Qué se perdió. Justificar con CAP.

| **Escenario** | **Qué se cayó** | **Qué siguió funcionando** | **Qué se perdió** | **Justificación CAP / PACELC / ACID-BASE** |
|---|---|---|---|---|
| **a) Postgres detenido** | PostgreSQL (facturación ACID) | MongoDB (catálogo), Redis (carrito), Cassandra (reportes) | Emisión de facturas nuevas. Los datos ya persistidos no se pierden, pero no son accesibles mientras el contenedor esté detenido. | Postgres es **CP**: sin primario disponible rechaza escrituras para no comprometer consistencia. PACELC: en partición **PC/EC** (prefiere consistencia sobre disponibilidad); sin partición **EC** (consistencia sobre latencia). Redis y Cassandra son **AP/BASE** y siguen disponibles. |
| **b) Mongo detenido** | MongoDB (catálogo polimórfico) | PostgreSQL (facturación) , Redis (carrito), Cassandra (reportes) | Lectura/escritura del catálogo de productos. La facturación sigue operando porque vive en otro motor. | MongoDB en Replica Set es **CP**: sin primario no acepta operaciones. PACELC: **PC/EC**. Postgres sigue siendo **CP** y disponible para facturar. Redis y Cassandra son **AP/BASE**. |
| **c) Redis detenido** | Redis (carrito, sesiones, caché) | PostgreSQL (facturación) , MongoDB (catálogo), Cassandra (reportes) | Carritos activos en memoria. Al ser datos efímeros, se pierden; al levantar Redis, el AOF recupera el último estado persistido ("TV,Arroz"). | Redis es **AP/BASE**: prioriza disponibilidad y baja latencia; consistencia eventual. PACELC: **PA/EL**. Su caída no afecta la transacción principal porque el stock real y las facturas están en Postgres. |

| | | | | |
|---|---|---|---|---|
| **d) Cassandra detenido** | Cassandra (búsquedas y reportes históricos) | PostgreSQL (facturación) , MongoDB (catálogo), Redis (carrito) | Registro de nuevas búsquedas/reportes . Los reportes ya escritos no se pierden, pero no son accesibles mientras el nodo esté caído. | Cassandra es **AP/BASE**: tolerante a particiones y con consistencia ajustable (QUORUM). PACELC: **PA/EL**. Su caída no afecta la operación transaccional ni el catálogo. |

## Video defensa:

https://drive.google.com/file/d/1W7wNTwNavIp2UZgJi4TyLDESpvJQg0_0/view?usp=sharing

## 5. REFERENCIAS


- Apache Software Foundation. (s. f.). *Apache Cassandra 4.1 documentation: Dynamo architecture*. https://cassandra.apache.org/doc/4.1/cassandra/architecture/dynamo.html
- MongoDB, Inc. (s. f.). *Replication*. MongoDB Manual. https://www.mongodb.com/docs/manual/replication/
- PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation: High availability, load balancing, and replication*. https://www.postgresql.org/docs/16/high-availability.html
- Pritchett, D. (2008). BASE: An ACID alternative. *ACM Queue, 6*(3), 48–55. https://doi.org/10.1145/1394127.1394128
- Redis Ltd. (s. f.). *Redis replication*. https://redis.io/docs/latest/operate/oss_and_stack/management/replication/